

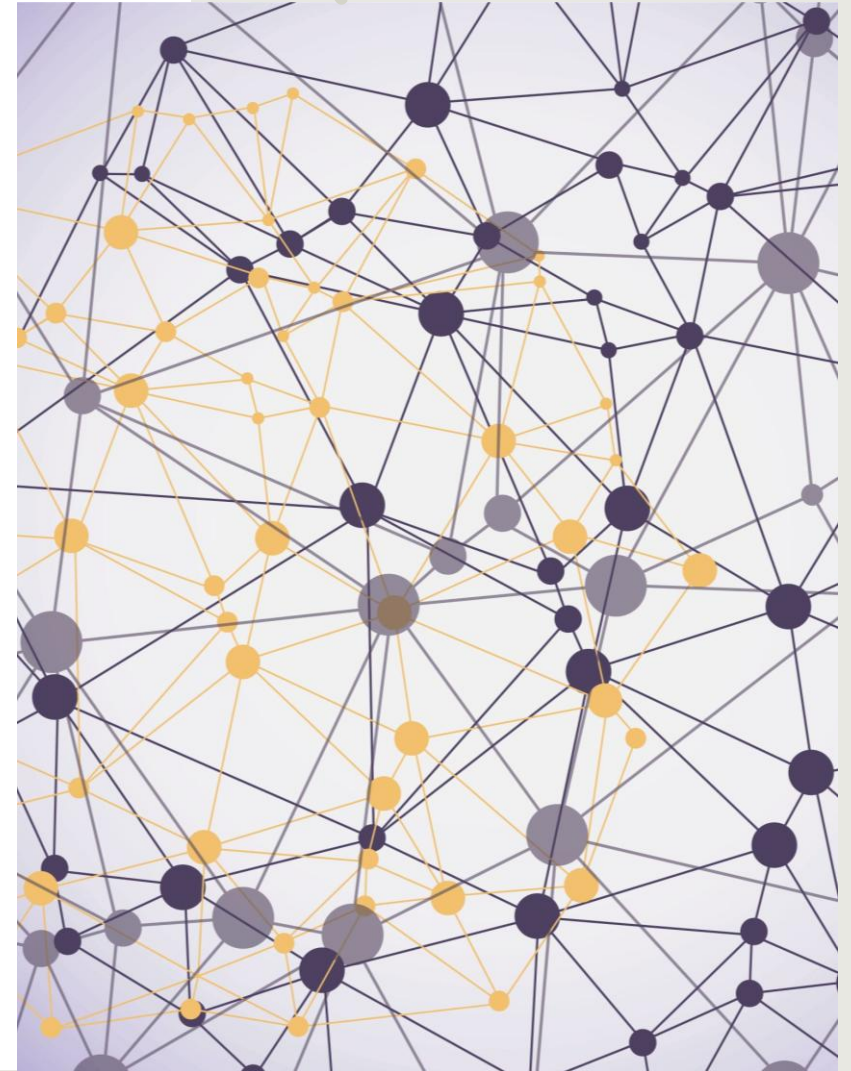
PACHETE SOFTWARE

SEMINAR 1

Recapitulare Python – Noțiuni introductive

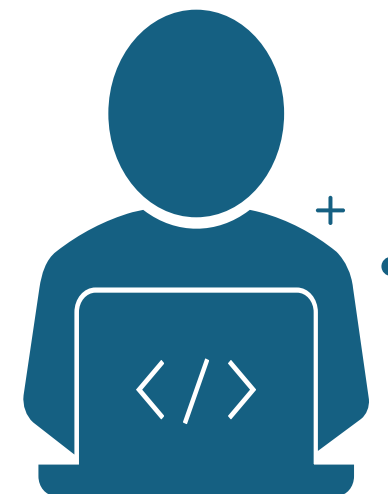
Elemente de programare în limbajul Python:

- ✓ **TIPURI DE DATE,**
- ✓ **VARIABLE,**
- ✓ **OPERATORI,**
- ✓ **LISTE,**
- ✓ **DICȚIONARE,**
- ✓ **SETURI,**
- ✓ **TUPLURI,**
- ✓ **STRUCTURI ALTERNATIVE ȘI REPETITIVE.**



Exercițiul 1:

- Declară și inițializează două variabile: nume (șir de caractere) și varsta (număr întreg).
- Citește un al treilea număr de la tastatură, care va fi folosit la diverse calcule.
- Afișează un mesaj de salut care conține atât nume, cât și varsta.
- Verifică tipurile variabilelor folosind `type()` și afișează rezultatul.
- Efectuează operații aritmetice (adunare, scădere, înmulțire) între varsta și numărul citit de la tastatură.
- Transformă varsta într-un șir de caractere și demonstrează ce se întâmplă dacă încerci să o folosești ulterior într-o operație matematică.
- Tratează erorile: dacă valoarea introdusă de la tastatură nu este un număr valid, gestionează excepția și afișează un mesaj corespunzător.



#Exercitiul 1

1. Declararea și inițializarea variabilelor

```
nume = "Maria"
```

```
varsta = 28
```

2. Citirea unui al treilea număr de la tastatură

(încercăm să-l convertim la int și tratăm posibile erori)

```
try:
    user_input = input("Introdu un număr pentru operații: ")
    numar_tastatura = int(user_input)
except ValueError:
    print("Valoarea introdusă nu este un număr valid. Se va folosi numărul 0.")
    numar_tastatura = 0
```

3. Afișarea unui mesaj de salut cu nume și varsta

```
print(f"Salut, {nume}! Ai {varsta} ani.")
```

f-string-uri (recomandat în Python 3+)

F-string-urile oferă o modalitate foarte intuitivă de a îmbina variabilele cu textul

4. Verificarea tipurilor variabilelor

```
print(f"Tipul variabilei 'nume' este: {type(nume)}")
```

```
print(f"Tipul variabilei 'varsta' este: {type(varsta)}")
```

```
print(f"Tipul variabilei 'numar_tastatura' este: {type(numar_tastatura)}")
```

5. Operații aritmetice între varsta și numărul introdus

```
print("\nOperatii aritmetice:")
suma = varsta + numar_tastatura
diferenta = varsta - numar_tastatura
produs = varsta * numar_tastatura
print(f"{varsta} + {numar_tastatura} = {suma}")
print(f"{varsta} - {numar_tastatura} = {diferenta}")
print(f"{varsta} * {numar_tastatura} = {produs}")
```

6. Conversia variabilei 'varsta' la șir de caractere

```
varsta_str = str(varsta)
print(f"\n'Am convertit varsta la șir: '{varsta_str}' (tip: {type(varsta_str)})")
```

Încercăm să facem o operație numerică cu varsta_str (greșit, ar duce la eroare dacă nu convertim înapoi)

De exemplu:

try:

```
    rezultat_gresit = varsta_str + numar_tastatura # Aceasta produce o eroare
```

except TypeError as e:

```
    print("Eroare la operația între un șir și un număr:", e)
```

- ```
def google():
 x = "search"
 def bing():
 x = "ads"
 bing()
 return x

print(google())
```

- ```
name = "python"
if name == "java" or "html":
    print("Login Successful")
else:
    print("Authentication Failed")
```

- cities = {
 ["Amsterdam"]: 100,
 ["Tokyo"]: 37400068,
 ["New York"]: 18819000,
 ["London"]: 9304016,
}


```
i = 1
```

```
while i < 0:
```

```
    print(i)
```

```
else:
```

```
    print("Done")
```

Definirea variabilelor

```
x = 'I am a string'  
a = 5    # int  
y = x    # string  
z = 2.36  # float  
n = 4j    # complex
```

Reguli pentru numele variabilelor

01

1. Pot începe doar cu o literă sau cu caracterul _

02

2. Sunt case-sensitive. Variabila num este diferită de variabila NUM și diferită de variabila Num

03

3. Nu pot fi folosite cuvinte rezervate din limbajul Python

04

4. De recomandat – să se folosească nume sugestive

Tipuri de date

Numeric - pentru date utilizate în calcule

Șiruri de caractere - pentru caractere text și pentru date binare. (String)

Secvențele de date - pentru colecții de date pe care le putem sorta, combina.

Dicționarele- colecții de date care asociază o cheie unică fiecărei valori.

Seturile sunt pentru operații aferente mulțimilor (găsirea intersecțiilor, diferențelor, etc.)

Fișierele sunt pentru date care sunt sau vor fi stocate ca document pe un computer cu mai multe valori.

Tipuri de date. Tipul numeric

- **INT**

Exemplu: 2,50,120

FLOAT

Exemplu: 1.23, 2.358972

COMPLEX:

Exemplu: 3+4j

Tipuri de date. Metoda type()

- În Python, se poate determina tipul de date al oricărui obiect folosind funcția **type()**.

```
4      z = 2.36 # float
5      n = 4j   # complex
6      print(type(2.36))
7      print(type(z))
8      print(type(n))
```

```
<class 'float'>
```

```
<class 'float'>
```

```
<class 'complex'>
```

Metode Built-In în Python

-
- Ce sunt metodele built-in?
 - Funcții și metode disponibile implicit în Python, fără a fi nevoie de importuri suplimentare, una dintre ele fiind metoda `type()`
 - Sunt optimizate pentru performanță și ușurință în utilizare.

Exemple:

- **`print()`** – Afișează mesaje în consolă.
- **`len()`** – Returnează lungimea unui obiect (șir, listă, etc.).
- **`type()`** – Afișează tipul unui obiect.
- **`input()`** – Citește input-ul de la utilizator.
- **`sorted()`, `sum()`, `min()`, `max()`** – Funcții de agregare și sortare.

Tipuri de date. Dynamic Typing

- Tipul unei variabile este determinat automat în timpul execuției programului, nu în momentul scrierii codului.
- Acest lucru înseamnă că nu trebuie specificat explicit tipul de date al unei variabile în momentul declarării.

```
x = 2
print(type(x))
x = " what type am I? "
print(type(x))
```

```
<class 'int'>
<class 'str'>
```

Tipuri de date. NoneType

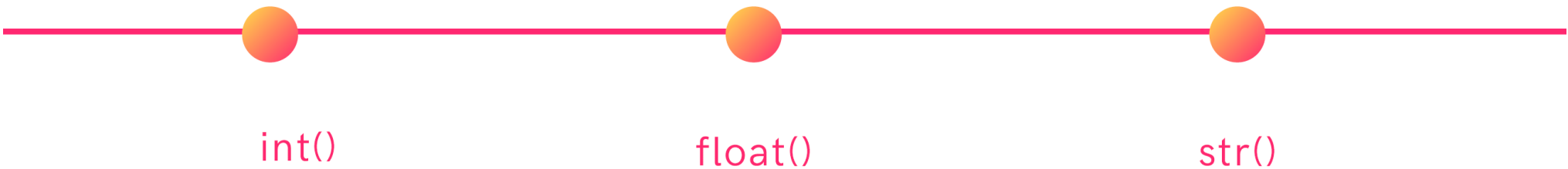
- NoneType este tipul de date asociat cu valoarea **None**, folosit pentru a reprezenta absența unei valori.

Tipuri de date. Conversia tipurilor de date

Python un limbaj orientat pe obiecte și folosește clase pentru a defini toate tipurile sale de date.

Fiecare tip de date în Python este de fapt o instanță a clasei respective.

Python folosește funcții constructor ale acestor clase pentru conversia tipurilor de date.



Tipuri de date. Conversia tipurilor de date

int()

```
x = int(1)    # 1
y = int(2.8)  # 2
z = int("3")  # 3
print(x, y, z)
print(type(x), type(y), type(z))
```

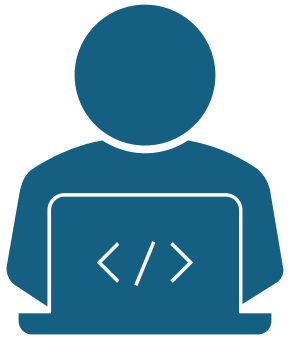
float()

```
x = float(1)    # 1.0
y = float(2.8)  # 2.8
z = float("3")  # 3.0
w = float("4.2") # 4.2
```

str()

```
x = str("s1")   # 's1'
y = str(2)      # '2'
z = str(3.0)    # '3.0'
```

Exercițiul 2



Scrie un program în Python care:

- Creează un șir de caractere cu mesajul: "Bine ai venit la cursul de Python".
- Afișează primul caracter al șirului.
- Afișează subșirul format din caracterele de la poziția 3 până la 7 (folosind slicing).
- Transformă întregul mesaj în litere mari și afișează-l.

Declararea șirului

```
mesaj = "Bine ai venit la cursul de Python"
```

Afișarea primului caracter

```
print("Primul caracter:", mesaj[0])
```

Extrage și afișează subșirul de la poziția 3 la 7

```
print("Subșirul de la poziția 3 la 7:", mesaj[3:8])
```

Transformă șirul în litere mari și afișează-l

```
mesaj_mare = mesaj.upper()
```

```
print("Mesajul în litere mari:", mesaj_mare)
```

Tipuri de date secvențiale

- Tipurile de date secvențiale în limbajele de programare precum Python sunt proiectate pentru a stoca mai multe elemente într-o manieră ordonată, unde fiecărui element i se atribuie o poziție sau un index specific în cadrul secvenței.

Tipuri de date secvențiale.

- **Listele** pot stoca mai multe tipuri de date (atat text cât și numere, de exemplu).
- **Tuplurile**, la fel ca listele, pot include diferite tipuri de date, dar nu pot fi modificate.
- **Șirurile de caractere** (Strings) stochează text sau date binare. Șirurile de caractere sunt nemodificabile (la fel ca tuplurile).

Tipuri de date secvențiale. String

- - suportă ordonare în funcție de poziție
- -extragerea unui caracter de pe o anumită poziție se poate face prin **indexare**.

```
a = "Hello World"  
print(a[0])
```

```
x = "supercalifragilisticexpialidocious"  
print(len(x))
```

Tipuri de date secvențiale. String

```
a = "Hello World"  
print(a[-1])  
print(a[-2])
```

Tipuri de date secvențiale. String

- În plus față de indexarea pozițională simplă, secvențele suportă și o formă mai generală de indexare cunoscută sub numele de secționare (**slicing**), care este o metodă de extragere a unei întregi secțiuni (felie) într-un singur pas.

- De exemplu:

```
a = "Hello"  
print(a[1:3])
```

Tipuri de date secvențiale. String

- Secționările (slices) sunt o metodă de a extrage o întreagă coloană dintr-un șir de caractere într-un singur pas.
- Forma lor generală, **X[I:J]**, înseamnă „preia totul din X de la indexul I până la, **dar fără a include**, indexul J”.
- Într-o secționare, limita stângă are valoarea implicită zero, iar limita dreaptă are valoarea implicită lungimea secvenței care se secționează

```
print(a[1:])  
print(a[0:3])  
print(a[:3])  
print(a[:-1])  
print(a[:])
```



În exemplele anterioare, nu modificam șirul original cu niciuna dintre operațiunile pe care le efectuam asupra acestuia.

Fiecare operație cu șiruri de caractere este definită pentru a produce un nou șir ca rezultat, deoarece șirurile de caractere sunt **nemodificabile** în Python.

```
a[0] = 'b'
```

```
a[0] = 'b'
```

```
TypeError: 'str' object does not support item assignment
```

```
d
```

Metode specifice clasei String

```
a = "Come on everybody"
everybody = a.find('everybody')
print(everybody)

b = a.replace('everybody', 'Tom')
print(b)
```

Metode specifice clasei String

```
line = 'aaa,bbb,ccccc,dd'
line.split(',') # Split on a delimiter into a list of substrings
S = 'My string'
S.upper() # Upper- and lowercase conversions
S.isalpha() # Content tests: isalpha, isdigit, etc.
line = 'w o w '
line.strip()
line = 'aaa, bbb, ccccc,dd\n'
line = line.rstrip() # Remove whitespace characters on the right side
```

POLIMORFISM



```
print(a + 'Everybody')  
print(a * 10)
```

- Șirurile de caractere suportă **concatenarea** cu semnul plus (+) (unind două șiruri de caractere într-un șir nou) și repetiția (creând un șir nou prin repetarea altuia):
- Semnul plus (+) înseamnă lucruri diferite pentru obiecte diferite: adunare pentru numere și concatenare pentru șiruri.
- Acesta este o proprietate generală a limbajului Python numită **polimorfism** - semnificația unei operații depinde de obiectele asupra cărora se efectuează operația.



Exercițiul 3: Lucrul cu liste

Scrie un program în Python care:

- Creează o listă cu 5 orașe,
- afișează lungimea acesteia,
- adaugă un nou oraș,
- elimină primul element și sortează lista în ordine alfabetică.

```
# Crearea listei de orașe  
orase = ["București", "Cluj", "Timișoara", "Iași", "Constanța"]
```

```
# Afișarea lungimii listei  
print("Lungimea listei:", len(orase))
```

```
# Adăugarea unui nou oraș  
orase.append("Brașov")  
print("Lista după adăugare:", orase)
```

```
# Ștergerea primului element  
del orase[0]  
print("Lista după ștergerea primului element:", orase)
```

```
# Sortarea listei în ordine alfabetică  
orase.sort()  
print("Lista sortată:", orase)
```

Tipuri de date secvențiale. LISTE



colecții de obiecte cu tipuri arbitrare, ordonate în funcție de poziție



nu au o dimensiune fixă



sunt modificabile



-fiind tot secvențe, listele suportă toate operațiile de secvență discutate anterior pentru șirurile de caractere (indexare, partiționare); singura diferență este că rezultatele sunt, de obicei, liste în loc de șiruri de caractere."

This is your
"movies" list in
code.

Item #0

Item #1

Item #2

```
movies = ["The Holy Grail", "The Life of Brian", "The Meaning of Life"]
```

This is your "movies"
list in memory.

"The Meaning of Life"

2

"The Life of Brian"

1

"The Holy Grail"

0

Each data item
in the list has a
numeric OFFSET
associated with it.

Python starts counting
from zero.

Tipuri de date secvențiale. LISTE

```
movies = ["The Holy Grail", "The Life of Brian", "The Meaning of Life"]
print(len(movies)) # number of items in the list
first_movie = movies[0] # Indexing by position
print(first_movie)
print(movies[:-1]) # Slicing a list returns a new list
print(movies + ['Titanic']) # Concatenation makes a new list too
```

LISTE: Operații specifice

```
movies.append(1.23)  #Growing: add object at end of list
movies.append(5)
print(movies)
movies.pop(2)  #Shrinking: delete the third element
print(movies)
movies.pop()  #delete the last element
print(movies)
movies.append('Alice in Wonderland')  #Growing: add object at end of list
print(movies)
del movies[2]  #deletes from a list
print(movies)
```

Tipuri de date. Tupluri

Tuplurile sunt secvențe, asemenea listelor, dar sunt imutabile, asemenea șirurilor de caractere. Ele suportă tipuri arbitrare, și operațiile obișnuite pentru secvențe.

```
T = (0, 1, 2, 3)
print(len(T)) # number of elements
print(T + (5, 6, 7, 8)) # concatenation
print(T[0]) # indexing & slicing
print(T[0:3])
```

Tupluri. Operații specifice

```
T = (0, 1, 2, 3)
T = T + (5, 6, 7, 8)
print(T.index(5)) #element 5 has index 4
print(T.count(6)) #6 appears once
```


Exercițiul 4: Manipularea dicționarelor

Scrie un program în Python care:

- Creează un dicționar cu produse și prețuri (ex.: {"telefon": 1500, "laptop": 3500, "tabletă": 1200}).
- Afișează prețul produsului "telefon" folosind cheia.
- Adaugă un nou produs ("smartwatch" cu prețul 800).
- Actualizează prețul produsului "laptop" la 3600. Parcurge dicționarul și afișează fiecare pereche cheie-valoare.

#ExercitiuL 5

```
produse = {"telefon": 1500, "laptop": 3500, "tabletă": 1200}
```

Afișarea prețului pentru "telefon"

```
print("Prețul telefonului:", produse["telefon"])
```

Adăugarea unui nou produs

```
produse["smartwatch"] = 800
```

```
print("Dicționarul după adăugarea smartwatch-ului:", produse)
```

Actualizarea prețului pentru "laptop"

```
produse["laptop"] = 3600
```

```
print("Dicționarul după actualizarea prețului laptopului:", produse)
```

Parcurgerea dicționarului și afișarea perechilor

```
for produs, pret in produse.items():
```

```
    print(f"Produs: {produs}, Preț: {pret}")
```


Tipuri de date. Dicționare

Dicționarul în Python (cu cuvânt-cheie 'dict') este un tip de date care stochează mai multe elemente de date de diferite tipuri.



Atunci când utilizați un dicționar, utilizați cheia pentru a obține elementul asociat cu cheia respectivă.



Deoarece dicționarele nu sunt secvențe, ele nu mențin nicio ordine de la stânga la dreapta pe care să te poți baza.



Acest lucru înseamnă că atunci când creăm un dicționar și îl afișăm, cheile sale pot reveni într-o ordine diferită față de cea în care le-am tastat.

Dicționare. Operații specifice

```
x = ('key1', 'key2', 'key3')
y = 0
thisdict = dict.fromkeys(x, y) # Returns a dictionary with the specified keys and value
print(thisdict)

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.items() # Return the dictionary's key-value pairs
print(x)
print(car.keys()) # Returns a list containing the dictionary's keys
```

Dicționare. Operații specifice

```
y = car.setdefault("model", "Bronco") # Returns the value of the specified key.  
print(y) # If the key does not exist: insert the key, with the specified value  
car.update({"color": "White"}) # Insert an item to the dictionary  
print(car)  
v = car.values()  
print(v) # Returns a list of all the values in the dictionary  
car.pop("model") # Removes the element with the specified key  
print(car)  
car.popitem() # Remove the last item from the dictionary  
print(car)
```

Tipuri de date. SETURI

Python 2.4 a introdus, de asemenea, un nou tip de colecție, setul - o colecție **neordonată** de obiecte unice, care suportă operații corespunzătoare teoriei matematice a mulțimilor.

Prin definiție, un element apare **doar o singură dată** într-un set, indiferent de câte ori este adăugat. De aceea, seturile au o varietate de aplicații, în special în lucrul cu numere și în baza de date.

```
SET = {1, 2, 3, 4, 5}
set_string = set('python')
print(SET)
print(set_string) # the display is random, because the elements are unordered
```

SETURI. Operații specifice

```
engineers = {'bob', 'susan', 'mary'}  
managers = {'jake', 'monica', 'mary'}  
  
#1. Is Bob an engineer?  
print('bob' in engineers)  
|  
  
#2. Who is both an engineer and a manager?  
print(engineers.intersection(managers))  
  
#or  
print(engineers & managers)  
  
#3. All the employees are:  
print(engineers.union(managers))  
  
#or  
print(engineers | managers)
```


SETURI. Operații specifice

```
#4. Who is an engineer but not a manger?  
print(engineers.difference(managers))  
#or  
print(engineers.symmetric_difference(managers))  
#or  
print(engineers - managers)  
  
#5. Are all managers engineers (superset)?  
print(managers.issuperset(engineers))  
#or  
print(managers > engineers)  
  
#6. Are Bob and Susan engineers (subset)?  
print({'bob', 'susan'}.issubset(engineers))  
#or  
print({'bob', 'susan'} < engineers)
```

<code>x or y:</code>	• Returnează True dacă x sau y este True.
<code>x and y:</code>	• Returnează True dacă atât x cât și y sunt True.
<code>not x:</code>	• Inversează valoarea de adevăr, returnează True dacă x este False și invers.
<code>x in y:</code>	• Returnează True dacă x este un membru al lui y (de exemplu, o listă sau un șir).
<code>x not in y:</code>	• Returnează True dacă x nu este un membru al lui y.
<code>x is y:</code>	• Identitate - Returnează True dacă x și y sunt același obiect.
<code>x is not y:</code>	• Identitate - Returnează True dacă x și y nu sunt același obiect.
<code>x < y:</code>	• Mai mic decât - Returnează True dacă x este mai mic decât y.
<code>x <= y:</code>	• Mai mic sau egal cu - Returnează True dacă x este mai mic sau egal cu y.
<code>x > y:</code>	• Mai mare decât - Returnează True dacă x este mai mare decât y.
<code>x >= y:</code>	• Mai mare sau egal cu - Returnează True dacă x este mai mare sau egal cu y.
<code>x == y:</code>	• Egal cu - Returnează True dacă x este egal cu y.
<code>x != y:</code>	• Diferit de - Returnează True dacă x nu este egal cu y.
<code>x y:</code>	• SAU pe biți - Efectuează operația OR pe fiecare pereche de biți.
<code>x ^ y:</code>	• XOR pe biți - Efectuează operația XOR exclusiv pe fiecare pereche de biți.
<code>x & y:</code>	• ȘI pe biți - Efectuează operația AND pe fiecare pereche de biți.
<code>x << y:</code>	• Deplasare la stânga pe biți - Deplasează biții lui x la stânga cu y poziții.
<code>x >> y:</code>	• Deplasare la dreapta pe biți - Deplasează biții lui x la dreapta cu y poziții.
<code>x + y:</code>	• Adunare - Adună x și y.
<code>x - y:</code>	• Scădere - Scade y din x.

Exercițiul 5: Structuri repetitive și condiționale

- Scrie un program care:
- Parcurge o listă de numere de la 1 la 10.
- Afișează doar numerele pare.
- Bucla de parcurgere trebuie să sară peste numerele impare.
- Opțional: oprește iterarea când întâlnește numărul 8 folosind break.

Crearea listei de numere de la 1 la 10

```
numere = list(range(1, 11))
```

```
for numar in numere:
```

```
    # Verificăm dacă numărul este impar și trecem la următoarea iterație
```

```
    if numar % 2 != 0:
```

```
        continue
```

```
    # Dacă numărul este egal cu 8, ieșim din buclă (opțional)
```

```
    if numar == 8:
```

```
        print("Numărul 8 întâlnit, întrerupem bucla.")
```

```
        break
```

```
    # Afișăm numărul par
```

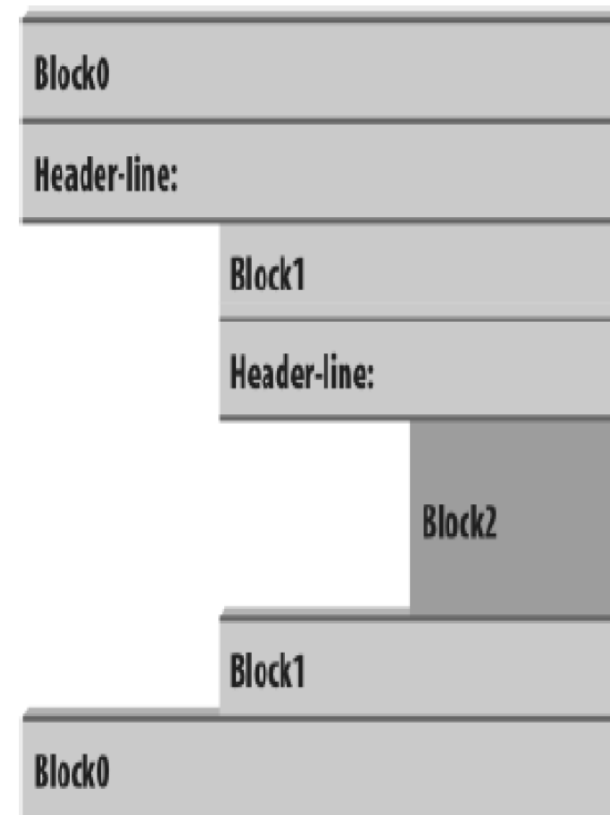
```
    print("Număr par:", numar)
```

STRUCTURI ALTERNATIVE

```
if <test1>:  
    <statements1>  
elif <test2>:  
    <statements2>  
else:  
    <statements3>
```

Reguli de indentare

```
x = 1
if x:
    y = 2
    if y:
        print('block2')
    print('block1')
print('block0')
```



STRUCTURI ALTERNATIVE

```
print("Hello");  
number = input("Make a guess") #  
guess = int(number);  
if guess == 26:  
    print("WIN!")  
else:  
    print("YOU LOSE! TRY AGAIN!")  
print("Game over")
```

STRUCTURE REPETITIVE

```
while <test>:                # Loop test
    <statements1>            # Loop body
else:                        # Optional else
    <statements2>            # Run if didn't exit loop with break
```


STRUCTURI REPETITIVE

```
a = 0
b = 10
while a < b: # One way to code counter loops
    print(a, end=' ')
    a += 1 # Or, a = a + 1
```

STRUCTURI REPETITIVE

- **break**: Când o instrucțiune break este întâlnită în interiorul unei bucle, bucla este imediat terminată și controlul programului continuă la următoarea instrucțiune imediat după buclă.
- **continue**: Când o instrucțiune continue este întâlnită în interiorul unei bucle, iterația curentă a buclei este terminată, și controlul programului revine la începutul buclei, la linia antet a buclei, unde condiția este reevaluată.

STRUCTURE REPETITIVE

```
while <test1>:  
    <statements1>  
    if <test2>: break           # Exit loop now, skip else  
    if <test3>: continue       # Go to top of loop now, to test1  
else:  
    <statements2>             # Run if we didn't hit a 'break'
```

STRUCTURI REPETITIVE

```
x = 10
while x:
    x = x-1  # Or, x -= 1
    if x % 2 != 0: continue  # Odd? -- skip print
    print(x, end=' ')
```

STRUCTURI REPETITIVE

```
x = 10
while x:
    x = x-1 # Or, x -= 1
    if x % 2 == 0: break
    print(x, end=' ')
```

STRUCTURI REPETITIVE

- În Python, bucla for începe cu o linie de antet care specifică o variabilă țintă (sau mai multe variabile ținte), împreună cu obiectul pe care doriți să îl parcurgeți. Antetul este urmat de un bloc de instrucțiuni (de obicei indentate) pe care doriți să le repetați. Iată structura generală a unei bucle for în Python:

```
x = ['pizza', 'cheese', 'ham']  
for food in x:  
    print(food)
```

Funcții

```
def nume_functie(parametru1, parametru2=valoare_implicita):  
    """  
    Documentația funcției (docstring)  
    Explică ce face funcția și parametrii  
    """  
    # Bloc de cod  
    # Returnează o valoare opțional (folosind `return`)
```

def este cuvântul cheie folosit pentru a defini o funcție.

nume_functie trebuie să fie un identificator valid (nu poate începe cu cifre, nu poate conține spații etc.).

`parametru2=valoare_implicita` definește un parametru opțional, cu o valoare implicită dacă nu este furnizată alta.

Docstring-ul (între `""" ... """`) descrie funcția și parametrii. Poate fi accesat ulterior cu `help(nume_functie)`.

Funcții Lambda

- Funcții anonime, scrise pe o singură linie, fără cuvântul cheie def.
- De obicei folosite pentru operații scurte, când nu e necesar un nume formal de funcție.

`lambda param1, param2, ...: expresie`

- Nu se folosesc instrucțiuni return; valoarea întoarsă este rezultatul expresiei.

Funcție lambda care adună două numere

`suma = lambda x, y: x + y`

`print(suma(3, 5))` # Output: 8

Avantaje și utilizări:

- ✓ Scurte și concise: pentru expresii simple.
- ✓ Flexibile: Folosite adesea cu funcții built-in ca `map()`, `filter()`, `sorted()` (prin key).
- ✓ Limitare: Nu pot conține instrucțiuni complexe (ex. `if/else` extins, bucle etc. pot fi folosite doar sub formă de expresie).

TEMA 1

Ai o evidență a angajaților unei companii, unde fiecare angajat este reprezentat ca un tuplu cu informațiile: (ID, nume complet, departament, salariu)

Realizează următoarele operații:

1. Creează o listă de angajați (tupluri) predefinită.
2. Scrie o funcție care primește lista de angajați și un nume de departament și returnează o listă cu angajații din acel departament.
3. Scrie o funcție care calculează salariul mediu al angajaților.
4. Scrie o funcție care identifică angajatul cu cel mai mare salariu și cel cu cel mai mic salariu.
5. Scrie o funcție care sortează angajații după salariu (de la cel mai mic la cel mai mare) și returnează o nouă listă de tupluri sortate.

- Listă de angajați: fiecare angajat este un tuplu (ID, nume complet, departament, salariu)

```
angajati = [  
    (1, "Popescu Ion", "IT", 5000),  
    (2, "Ionescu Maria", "HR", 4500),  
    (3, "Georgescu Mihai", "IT", 6000),  
    (4, "Dumitru Ana", "Finance", 7000),  
    (5, "Stanciu Elena", "HR", 4000),  
    (6, "Marinescu Vlad", "IT", 5500)  
]
```

```
# Funcția pentru selectarea angajaților dintr-un departament dat  
def selecteaza_dupa_departament(lista_angajati, departament):  
    return [angajat for angajat in lista_angajati if angajat[2] == departament]
```

Funcția pentru calcularea salariului mediu

```
def calculeaza_salariu_mediu(lista_angajati):  
    total_salarii = sum(angajat[3] for angajat in lista_angajati)  
    return total_salarii / len(lista_angajati) if lista_angajati else 0
```

Funcția pentru identificarea angajatului cu cel mai mare și cel mai mic salariu

```
def extrem_salarii(lista_angajati):  
    # Comparăm după salariu, care se află la indexul 3  
    angajat_max = max(lista_angajati, key=lambda ang: ang[3])  
    angajat_min = min(lista_angajati, key=lambda ang: ang[3])  
    return angajat_max, angajat_min
```

Funcția pentru sortarea angajaților după salariu

```
def sorteaza_dupa_salariu(lista_angajati):  
    return sorted(lista_angajati, key=lambda ang: ang[3])
```

TEMA 2

Ai o listă de comenzi pentru un magazin online. Fiecare comandă este reprezentată printr-o listă care conține următoarele elemente, în această ordine:

1. **ID-ul comenzii** (număr întreg)
2. **Numele clientului** (șir de caractere)
3. **Lista de produse comandate** (listă de șiruri de caractere)
4. **Totalul comenzii** (număr zecimal)
5. **Statusul comenzii** (șir de caractere) – de exemplu: "Finalizată", "În curs" sau "Anulată"

Extrage toate comenzile care au statusul "Finalizată".

Calculează suma totală a comenzilor care sunt "Finalizate".

Găsește comanda cu cel mai mare total de plată.

Sortează comenzile în funcție de totalul comenzii, în ordine crescătoare.

Schimbă statusul comenzii cu un anumit ID (de exemplu, ID-ul 3) la "Anulată".

Adaugă o nouă comandă în lista de comenzi.